🌱 Sprig-DB

Comprehensive Architectural Study Guide & Presentation Masterplan

Embedded Storage | ACID B+Tree Engine | Dynamic Schema Validation | SQL-on-NoSQL | Multi-Tenancy | HTMX Glassmorphism

---

Table of Contents

1. **Executive Presentation Pitch & Novelty** (What & Why)
2. **Concepts Learned & The Journey** (How Did You Learn It?)
3. **System Architecture & Transaction Pipeline** (Mental Model)
4. **Component-by-Component & Line-by-Line Code Breakdown**
    - Storage Core (`sprig/sprig.go`)
    - B-Tree Secondary Indexing (`sprig/index.go`)
    - Fluent Query Engine & Mutation Safety (`sprig/filter.go`)
    - Encoding & Number Fidelity (`sprig/encoding.go`)
    - Schema Engine & Preflight Dry-Run (`api/schema.go` & `api/schema_parser.go`)
    - SQL Parsing & AST Execution (`api/sql_query.go`)
    - Multi-Tenant Security & Captcha (`api/auth.go`)
    - REST Server & Atomic Bulk Ingestion (`api/api.go`)
    - Dashboard & Zero-Bundle Frontend (`api/web.go` & HTMX)
    - Benchmarking & Concurrency Proof (`cmd/loadtest/main.go`)
5. **Minute-by-Minute Presentation Script & Slide Outline**
6. **Tough Viva / Interview Defense Q&A**

---

1. Executive Presentation Pitch & Novelty

The Problem Statement

Modern application development is plagued by a binary choice between two extremes:

1. **Heavy Client-Server Databases (PostgreSQL, MongoDB):** Require Docker containers, networking overhead, connection pools, user provisioning, and maintenance ops. Total overkill for edge computing, local desktop tools, CLI utilities, and embedded devices.
2. **Primitive Embedded Key-Value Stores (raw BoltDB, RocksDB, SQLite):**
    - BoltDB and RocksDB only give you raw byte slices: `[]byte -> []byte`. You get zero document awareness, zero schema validation, no secondary index maintenance, and no query language.
    - SQLite is relational and fixed-schema; handling arbitrary polymorphic JSON structures requires clunky JSON extensions without native document collection abstractions.

> 💡 **The Novelty of Sprig-DB**
>
> **Sprig-DB bridges the gap between raw embedded key-value storage and full-fledged document databases.** It takes an ultra-reliable, zero-dependency embedded B+tree engine (`bbolt`) and builds a complete, production-ready document database platform directly in Go:
>
> - **Zero-Overhead Embedded Engine:** Runs in-process with your Go binary; single-file storage (`.sprig`), zero daemon processes, instant startup, and zero operational overhead.
> - **Automatic Secondary B-Tree Indexing:** Seamlessly extracts fields into secondary B-tree buckets using composite prefix keys (`value:id`), giving sub-millisecond equality queries without full table scans.
> - **Pre-Flight Schema Dry-Run (Novel Feature):** Unlike MongoDB (which validates only at insert time) or SQL (which locks tables during alters), Sprig-DB introduces an instant `/schema/check` dry-run endpoint. It evaluates a proposed schema against existing records in memory before committing, showing exact violating documents.
> - **Dual Query Engine (NoSQL & SQL):** Developers can query records either via a fluent programmatic Go API (`Coll("users").Eq(...).Find()`) or via standard SQL queries (`SELECT ... WHERE ... ORDER BY ... LIMIT ...`).
> - **Multi-Tenancy at the Storage Layer:** Strict tenant isolation (`_owner` partitioning) built straight into the engine, ensuring multi-user isolation on an embedded database file.
> - **Modern Glassmorphic Web Dashboard:** Server-rendered using Echo and HTMX—providing an administrative UI with zero JavaScript build toolchain (no Node, no Webpack).

---

2. Concepts Learned & The Learning Journey ("How Did I Learn It?")

In your presentation, tell this story chronologically. It demonstrates that you did not just copy code—you solved real systems-level engineering challenges:

Engineering Domain	Theoretical Concept	Practical Challenge & How You Learned It
Storage & ACID Engines	B+Tree layout, Memory-mapped I/O (mmap), Write-Ahead logging vs Copy-on-Write B-Trees.	Learned how `bbolt` achieves ACID guarantees using single-writer, multiple-reader transactions. Understood how data is organized into nested "Buckets" and persisted to a single memory-mapped file.
Binary Key Sorting	Big-Endian vs Little-Endian integer lexicographical sort order in B-Trees.	*The Bug:* When saving document IDs as Little-Endian or strings, ID `10` sorted before ID `2`. *The Solution:* Converted uint64 IDs into 8-byte Big-Endian byte arrays (`binary.BigEndian.PutUint64`), forcing the disk B-Tree to keep records in strictly ascending numerical order.
Secondary Indexing	Composite index keys, Inverted indexes, Range cursors, Prefix scanning.	Built index buckets formatted as `_idx_{coll}_{field}`. Designed composite keys `fmt.Sprintf("%v:%d", value, id)` so bbolt's cursor seek can jump directly to the target prefix in $O(\log N)$ time and iterate matches.
Concurrency & Synchronization	Read-Write Mutexes (`sync.RWMutex`), Transaction deadlocks, Goroutine safety.	BoltDB allows unlimited read transactions concurrently but strictly *one* write transaction at a time. Solved deadlocks by wrapping operations with granular mutexes and using `defer tx.Rollback()` before commit.
Data Fidelity & JSON Pitfalls	IEEE-754 64-bit float precision loss with large integers.	Standard Go `json.Unmarshal` turns every number into `float64`, truncating large int64 IDs. Solved by configuring custom decoders with `dec.UseNumber()` and building custom type coercers.
Query Parsing & Interpretation	Tokenization, Abstract Syntax Trees (AST), SQL execution pipelines.	Wrote a regular-expression and token-based parser for SQL (`SELECT`, `WHERE` with nested `AND` / `OR`, `ORDER BY`, `LIMIT`, `OFFSET`) that compiles into in-memory filters over document maps.
Defensive Mutation Safeguards	Fail-safe systems programming.	Implemented `ErrNoFilter`: prevented accidental execution of `UPDATE` or `DELETE` without a filter, preventing catastrophic data wipeouts.
High-Throughput Load Testing	Benchmarking, Concurrency barriers (`sync.WaitGroup`), Atomic counters.	Built a 200-worker concurrent HTTP load tester in `cmd/loadtest/main.go` blasting 25,000 schema-validated requests to verify engine stability under saturation.

3. High-Level System Architecture



---

4. Component-by-Component & Line-by-Line Code Breakdown

Module 1: `sprig/sprig.go` — The Core Engine & Lifecycle

This file defines the heart of the database: the `Sprig` struct, initialization, bucket creation, and database lifecycle management.

```
type Sprig struct {
    mu              sync.RWMutex  // Protects database handle & schema operations
    currentDatabase string        // Holds filename: e.g., "default.sprig"
    *Options                      // Embedded configuration (custom encoders/decoders)
    db              *bbolt.DB     // Underlying bbolt embedded database handle
}
```

Detailed Code Walkthrough:

- **Line 35:** `func New(options ...OptFunc) (*Sprig, error)`
  Implements the *Functional Options Pattern* in Go. Sets default options (standard JSON encoder/decoder, default DB name `"default"`), appends extension `.sprig`, and opens the file with permissions `0666` via `bbolt.Open()`.
- **Line 57:** `func (h *Sprig) Close() error`
  Acquires an exclusive write lock (`h.mu.Lock()`) to guarantee no concurrent queries or transactions are executing, then flushes and closes the bbolt file.
- **Line 76:** `func (h *Sprig) CreateCollection(name string) error`
  Begins an ACID write transaction: `tx, err := h.db.Begin(true)`.
  Crucial Go Pattern: `defer tx.Rollback()`. If any panic or error occurs before `tx.Commit()`, the transaction rolls back automatically with zero file corruption! If `tx.Commit()` succeeds, the rollback becomes a no-op.
- **Line 94:** `func (h *Sprig) DropCollection(name string) error`
  Deletes the primary collection bucket via `tx.DeleteBucket([]byte(name))`. Next, it scans all top-level buckets using `tx.ForEach` to find any index buckets matching the prefix `_idx_{collection}_` and deletes them too. This guarantees *zero orphan index buckets* on disk.
- **Line 132:** `func (h *Sprig) Coll(name string) *Filter`
  Returns a new `*Filter` query builder tied to the specified collection.
- **Line 137:** `func (h *Sprig) ListCollections() ([]string, error)`
  Acquires a read lock (`h.mu.RLock()`) and opens a read-only transaction (`h.db.View()`). Iterates over bucket names and filters out system buckets that start with `'_'` (such as `_schemas`, `_users`, `_logs`, and `_idx_*`), returning only user-visible collections.

Module 2: `sprig/index.go` — Secondary B-Tree Indexing

This is one of the most intellectually impressive components in Sprig-DB: turning raw key-value storage into an index-accelerated document store.

```
// Bucket Naming Convention:
// _idx_{collection}_{field}
//
// Key format inside index bucket:
// [field_value]:[document_id] -> value: [document_id (8 bytes)]
```

Key Mechanisms & Functions:

- **Sensitive Field Masking (Line 13):**
  `var sensitiveFields = map[string]bool{"password": true, "secret": true, "token": true}`.
  The indexing engine actively inspects incoming document fields and refuses to write index entries for passwords or secret tokens. This prevents credential leaks in index dumps.
- **Composite Index Key (Line 34):**
  `key := fmt.Appendf(nil, "%v:%d", value, id)`.
  Why composite `value:id`? In a secondary index, multiple documents can share the same field value (e.g. `role = "developer"`). If the key was merely the value, a new document would overwrite the previous document's key! By appending `:id`, every key is globally unique while staying sorted together.
- **Range Prefix Scanning (Line 63: `lookupByIndex`):**
  Takes the query value (e.g., `"active"`) and forms prefix `"active:"`. Uses bbolt's B-Tree cursor: `c.Seek(prefix)` jumps directly to the first match in $O(\log N)$ time! It then iterates sequentially (`c.Next()`) collecting document IDs until the key prefix changes.

Module 3: `sprig/filter.go` — Fluent Query Builder & Big-Endian Storage

Contains the query builder API, record serialization, big-endian key generation, and query execution.

1. Big-Endian Numerical Key Ordering:

```
func uint64Bytes(v uint64) []byte {
    b := make([]byte, 8)
    binary.BigEndian.PutUint64(b, v)
    return b
}
```

Why Big-Endian? B-Tree engines like bbolt sort keys using standard byte-by-byte lexicographical comparison (`bytes.Compare`). If you store numbers as Little-Endian or string representation, `"10"` comes before `"2"` . Big-Endian ensures binary byte order strictly matches numerical integer order (`0x00...01 < 0x00...02 < ...`).

2. Defensive Mutation Safeguards:

```
var ErrNoFilter = errors.New("refusing to mutate without filters — add at least one Eq() filter.
```

Both `Update()` (line 311) and `Delete()` (line 353) check `if len(f.compFilters) == 0 { return ErrNoFilter }` .
This stops developers from inadvertently running an empty update/delete that wipes out every document in production.

3. Smart Index Acceleration vs Full Scan:

- `Find()` (line 200) inspects active filters. If there is exactly one `Eq` filter on a single field, it executes `tryIndexFind()` to resolve records via the secondary B-tree index in milliseconds.
- If no index is available, it gracefully falls back to `findFiltered()` , streaming through the bucket cursor while evaluating predicate functions.

Module 4: `api/schema.go` & `api/schema_parser.go` — Schema Enforcement & Dry-Run

Implements document validation, nested objects/arrays, regex pattern matching, enum constraints, and the pre-flight dry-run validator.

The Schema Data Model:

```
type FieldSchema struct {
    Type             string                `json:"type"`             // string, number, bool, a
    Required         bool                  `json:"required"`
    Enum             []string              `json:"enum,omitempty"`
    Min              *float64              `json:"min,omitempty"`
    Max              *float64              `json:"max,omitempty"`
    MinLength        *int                  `json:"minLength,omitempty"`
    MaxLength        *int                  `json:"maxLength,omitempty"`
    Default          any                   `json:"default,omitempty"`
    AdditionalFields string                `json:"additionalFields,omitempty"`
    Properties       map[string]FieldSchema `json:"properties,omitempty"` // For nested objects
    Items            *FieldSchema          `json:"items,omitempty"`      // For arrays
}
```

The Pre-Flight Schema Dry-Run (`HandleCheckCollectionSchema`):

When an administrator edits a schema on a collection with thousands of existing records, standard databases either reject the alter or force a migration. Sprig-DB allows a **Dry-Run Verification**:

1. The handler receives the proposed schema without updating the database.
2. It iterates over all existing tenant documents in the collection.
3. It executes `ValidateDocument()` in memory on clones of the documents.
4. It counts exact matches, counts failures, and collects the first 10 violation error strings with document IDs.
5. The UI alerts the user: *"4 of 1,200 documents do not match the new schema"*, displaying the exact reasons!

Module 5: `api/sql_query.go` — SQL-on-NoSQL Parser & Execution Engine

Enables relational SQL queries over unstructured and semi-structured embedded JSON documents!

```
// Supported SQL Syntax:
// SELECT  FROM  [WHERE ] [ORDER BY  [ASC|DESC]] [LIMIT n] [OFFSET n]
var sqlSelectRe = regexp.MustCompile(
    `(?is)^SELECT\s+(.+?)\s+FROM\s+([A-Za-z_][A-Za-z0-9_]*)(?:\s+WHERE\s+(.+?))?(?:\s+ORDER\s+BY
)
```

Pipeline Stages of `ExecuteSQLQuery`:

1. **Clause Extraction:** Regexp decomposes the query into Select, From, Where, OrderBy, Limit, and Offset clauses.
2. **Tenant Filter:** Enforces `r["_owner"] == owner`, ensuring cross-tenant data leaks are physically impossible.
3. **Recursive Logical Evaluation (`evalWhereClause`):** Splits expressions on `OR`, then on `AND`. Evaluates comparison operators: `=`, `!=`, `>`, `<`, `>=`, `<=`, `LIKE` (substring pattern match), and `IN`.
4. **Multi-Key Stable Sorting (`sort.SliceStable`):** Parses comma-separated `ORDER BY age DESC, name ASC`, performing type-safe comparisons across strings and numbers.
5. **Pagination Window:** Slices the matching array according to `OFFSET` and `LIMIT`.
6. **Column Projection:** Filters out unwanted keys unless `SELECT *` was requested.

Module 6: `api/auth.go` — Multi-Tenancy & Visual SVG CAPTCHA

Provides enterprise multi-tenant isolation, bcrypt credential hashing, and zero-dependency CAPTCHA protection.

1. In-Memory Programmatic SVG CAPTCHA (Line 81):

Instead of pulling in bulky external image rendering libraries (CGo, libpng, freetype), Sprig-DB generates lightweight SVG vectors directly in Go! It generates random alphanumeric characters, creates noise vectors (angled crossing lines, randomized colored circles), base64-encodes the SVG as a `...` URI, and pairs it with a temporary token stored in a thread-safe `sync.Map`.

2. Multi-Tenant Partitioning:

Every user has their own distinct workspace. When user `alice` creates a collection or inserts documents, the database injects `_owner: "alice"` into the record metadata. Every search, update, delete, and list operation filters by `_owner`, guaranteeing total privacy on a shared embedded database file.

Module 7: `cmd/loadtest/main.go` — Concurrency & Stress Testing

Proves that the engine is not a toy, but a high-performance, robust system capable of enterprise throughput:

- Launches **200 concurrent worker goroutines**.
- Dispatches **25,000 HTTP requests** with JSON schema validation enabled.
- Utilizes `sync/atomic` counters for zero-contention metrics tracking.

- Demonstrates that BoltDB's single-writer architecture with Go's HTTP server handles thousands of requests per second with zero data corruption.

5. Minute-by-Minute Presentation Script & Slide Outline

Use this battle-tested 10-minute presentation structure. It highlights your technical depth and confidence:

Slide 1: Title & The Core Vision (Minute 0:00 - 1:30)

- *What to say:* "Good morning everyone. Today I'm presenting **Sprig-DB**, an ultra-lightweight, embedded document database with dynamic schema enforcement, SQL querying, and a modern glassmorphic dashboard built in Go. The motivation behind Sprig-DB was simple: developers building desktop tools, CLI tools, and edge services shouldn't have to spin up heavy MongoDB or Postgres daemons, but raw embedded key-value stores like BoltDB only give you raw bytes with zero validation, no secondary indexes, and no query language. Sprig-DB solves this by bringing document intelligence and schema safety to embedded storage."

Slide 2: Architecture & Storage Engine (Minute 1:30 - 3:30)

- *What to say:* "Under the hood, Sprig-DB is built on top of an ACID B+Tree engine using memory-mapped I/O. Our data layer organizes documents into collections using nested B-Tree buckets. One critical systems concept I tackled was **numerical key sorting in B-Trees**. B-Trees sort lexicographically. If document IDs are formatted as strings or Little-Endian integers, ID 10 sorts before ID 2. I solved this by encoding IDs as 8-byte Big-Endian byte slices, guaranteeing that physical disk order strictly matches numerical sequence order."

Slide 3: Secondary Indexing & $O(\log N)$ Queries (Minute 3:30 - 5:00)

- *What to say:* "To prevent full collection scans on queries, I engineered an automated secondary indexing engine. Whenever a document is inserted or updated, Sprig-DB writes composite keys formatted as `value:id` into dedicated index buckets. When an equality query arrives, the engine uses bbolt's cursor seek to jump to the prefix in $O(\log N)$ time and extracts matching IDs without scanning the entire collection. Furthermore, I implemented defensive indexing: sensitive fields like passwords, tokens, and secrets are automatically excluded from index buckets."

Slide 4: Dynamic Schemas & Pre-Flight Dry-Run (Minute 5:00 - 6:45)

- *What to say:* "The standout novelty of Sprig-DB is its **Dual Schema System and Pre-Flight Dry-Run verification**. We support both native schema definitions and industry-standard JSON-Schema Draft-04, enforcing field types, min/max ranges, string regex patterns, enums, and nested arrays/objects. Even better: in existing databases, altering a schema can silently corrupt data or fail midway. In Sprig-DB, we provide a `/schema/check` dry-run endpoint. Before applying a new schema, the engine validates all existing records in-memory, returning an exact audit of which documents fail and why, allowing administrators to make safe schema evolutions."

Slide 5: SQL-on-NoSQL & Multi-Tenant Security (Minute 6:45 - 8:15)

- *What to say:* "Sprig-DB includes an interactive dual query console: NoSQL programmatic filters and standard SQL queries. I built a SQL tokenizer and evaluator from scratch supporting column projection, WHERE clauses with boolean logic, multi-field ORDER BY, LIMIT, and OFFSET. On the security side, we have true multi-tenancy. All collections, schemas, queries, and request logs are partitioned per user via `_owner` metadata. We

also built an in-memory SVG CAPTCHA generator that protects authentication routes without external image dependencies."

Slide 6: Benchmarks, Demos & Conclusion (Minute 8:15 - 10:00)

- *What to say:* "To prove the production-readiness of the system, I authored a concurrent load tester that blasts 25,000 schema-validated requests across 200 concurrent workers, sustaining high throughput with zero transaction failures or race conditions. In summary, Sprig-DB proves that you don't need a heavy client-server database to have schemas, secondary indexing, SQL queries, and a beautiful UI. Thank you, and I am now ready for any questions!"

6. Tough Viva / Interview Defense Questions & Model Answers

Q1: Why did you choose bbolt instead of SQLite?
Answer: SQLite is relational and table-oriented; storing semi-structured polymorphic JSON documents in SQLite requires awkward JSON functions and circumvents relational indexing. BoltDB is a pure, minimalist B+Tree key-value store with clean transactional semantics. Building on bbolt gave us complete control over our document model, bucket hierarchies, custom secondary indexing, and dynamic schema validation layers.

Q2: BoltDB only allows a single concurrent writer. Doesn't that bottleneck write throughput?
Answer: BoltDB uses an MVCC single-writer, multiple-reader model. While writes are serialized, transactions commit directly to memory-mapped pages and disk sequentially, achieving tens of thousands of writes per second. Additionally, Sprig-DB features **Atomic Bulk Ingestion** (`InsertMany`), which commits up to 100 documents in a *single* write transaction, drastically reducing disk sync overhead and maximizing write efficiency.

Q3: How does your secondary index prevent key collisions for documents with identical values?
Answer: We format index keys as composite strings: `value:id` (for example, `engineer:101` and `engineer:102`). Because the primary document ID is globally unique, the composite key is guaranteed to be unique. This allows multiple documents to share the same indexed attribute value while occupying adjacent positions in the B-Tree for rapid prefix range scanning.

Q4: How do you prevent floating-point precision issues with large 64-bit integer IDs in JSON?
Answer: Standard Go `json.Unmarshal` decodes JSON numbers as `float64` , which causes precision loss for integers greater than 2^{53}. We configured our decoders with `dec.UseNumber()` and wrote custom numeric casting helpers (`toFloat64`) that preserve int64 fidelity during filter evaluations.

Q5: How does your SQL parser handle operator precedence between AND and OR?
Answer: In `api/sql_query.go` , we implement a two-tier evaluation strategy. The WHERE expression is first split along logical `OR` boundaries, and each segment is then split along logical `AND` boundaries. This naturally satisfies standard Disjunctive Normal Form (DNF), ensuring that `AND` binds tighter than `OR` during predicate evaluation.

Q6: What happens if the server crashes during an insert?
Answer: BoltDB is strictly ACID compliant. Every mutation runs inside a transaction block with `defer tx.Rollback()` . If a power outage or crash occurs before `tx.Commit()` successfully flushes to the write-ahead log and disk, the file remains completely uncorrupted in its last consistent state.

Q7: Why use HTMX instead of React or Vue for the dashboard?
Answer: HTMX aligns perfectly with the philosophy of an embedded database: simplicity and zero overhead. By rendering server-side templates with Go's `html/template` and letting HTMX handle partial DOM swaps, we achieved a dynamic, single-page-app feel with modal dialogs and live tables with *zero Node.js dependencies, zero bundle build steps, and instant load times*.
